

BlinkDB Storage Engine

1.0

Generated by Doxygen 1.9.8

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 BlinkDB Class Reference	5
3.1.1 Detailed Description	7
3.1.2 Constructor & Destructor Documentation	7
3.1.2.1 BlinkDB() [1/2]	7
3.1.2.2 BlinkDB() [2/2]	7
3.1.3 Member Function Documentation	7
3.1.3.1 compact() [1/2]	7
3.1.3.2 compact() [2/2]	8
3.1.3.3 del() [1/2]	8
3.1.3.4 del() [2/2]	9
3.1.3.5 flush() [1/2]	10
3.1.3.6 flush() [2/2]	10
3.1.3.7 get() [1/2]	10
3.1.3.8 get() [2/2]	11
3.1.3.9 search_sstables() [1/2]	12
3.1.3.10 search_sstables() [2/2]	13
3.1.3.11 set() [1/2]	13
3.1.3.12 set() [2/2]	14
3.1.4 Member Data Documentation	14
3.1.4.1 cache	14
3.1.4.2 capacity	15
3.1.4.3 size	15
3.1.4.4 sstables	15
3.1.4.5 store	15
3.2 BloomFilter Class Reference	15
3.2.1 Detailed Description	16
3.2.2 Constructor & Destructor Documentation	16
3.2.2.1 BloomFilter()	16
3.2.3 Member Function Documentation	16
3.2.3.1 add()	16
3.2.3.2 contains()	17
3.2.3.3 hash_i()	18
3.2.4 Member Data Documentation	18
3.2.4.1 bits	18
3.2.4.2 k_hashes	18
3.2.4.3 m_bits	18

3.3 BlinkDB::Entry Struct Reference	19
3.3.1 Detailed Description	19
3.3.2 Member Data Documentation	19
3.3.2.1 deleted	19
3.3.2.2 value	19
3.4 LRUCache< K, V > Class Template Reference	19
3.4.1 Detailed Description	20
3.4.2 Constructor & Destructor Documentation	20
3.4.2.1 LRUCache()	20
3.4.3 Member Function Documentation	21
3.4.3.1 get()	21
3.4.3.2 put()	21
3.4.3.3 remove()	22
3.4.4 Member Data Documentation	22
3.4.4.1 cacheList	22
3.4.4.2 cacheMap	22
3.4.4.3 capacity	23
3.5 RESPParser Class Reference	23
3.5.1 Detailed Description	23
3.5.2 Member Function Documentation	23
3.5.2.1 error()	23
3.5.2.2 ok()	24
3.5.2.3 parse()	24
3.5.2.4 serialize()	25
4 File Documentation	27
4.1 blinkdb.cpp File Reference	27
4.1.1 Function Documentation	28
4.1.1.1 binary_search_file()	28
4.1.1.2 build_index_for_sstable()	29
4.1.1.3 get_line_from_mapping()	30
4.1.1.4 map_file()	30
4.1.2 Variable Documentation	31
4.1.2.1 sstable_filters	31
4.1.2.2 sstable_indices	31
4.1.2.3 sstable_mmaps	31
4.2 blinkdb.h File Reference	31
4.3 blinkdb.h	32
4.4 Iru_cache.h File Reference	33
4.5 Iru_cache.h	33
4.6 server.cpp File Reference	34
4.6.1 Macro Definition Documentation	35

4.6.1.1 BUFFER_SIZE	35
4.6.1.2 MAX_EVENTS	36
4.6.1.3 PORT	36
4.6.2 Function Documentation	36
4.6.2.1 handleClientRequest()	36
4.6.2.2 main()	37
Index	39

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BlinkDB	BlinkDB is a lightweight database with an in-memory memtable and SSTable persistence . . .	5
BloomFilter	A simple Bloom filter implementation	15
BlinkDB::Entry	Structure to store a value and its deletion state	19
LRUCache< K, V >	A templated Least Recently Used (LRU) Cache	19
RESPParser	A simple RESP (REdis Serialization Protocol) parser and serializer	23

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

blinkdb.cpp	27
blinkdb.h	31
lru_cache.h	33
server.cpp	34

Chapter 3

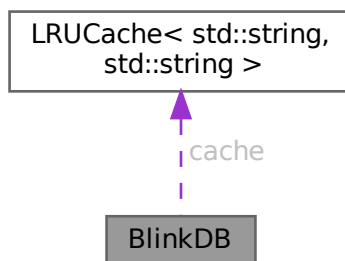
Class Documentation

3.1 BlinkDB Class Reference

[BlinkDB](#) is a lightweight database with an in-memory memtable and SSTable persistence.

```
#include <blinkdb.h>
```

Collaboration diagram for BlinkDB:



Classes

- struct [Entry](#)

Structure to store a value and its deletion state.

Public Member Functions

- [BlinkDB](#) (size_t [capacity](#)=64 *1024 *1024, size_t [cache_capacity](#)=16 *1024 *1024)
Constructs a [BlinkDB](#) instance.
- void [set](#) (const std::string &key, const std::string &value)
Sets a key-value pair in the database.
- std::string [get](#) (const std::string &key)
Retrieves the value associated with a given key.
- bool [del](#) (const std::string &key)
Deletes a key from the database.
- [BlinkDB](#) (size_t [capacity](#)=64 *1024 *1024, size_t [cache_capacity](#)=16 *1024 *1024)
Constructs a [BlinkDB](#) instance.
- void [set](#) (const std::string &key, const std::string &value)
Sets a key-value pair in the database.
- std::string [get](#) (const std::string &key)
Retrieves the value associated with a key.
- bool [del](#) (const std::string &key)
Deletes a key from the database.

Private Member Functions

- void [flush](#) ()
Flushes the memtable to an SSTable file.
- void [compact](#) ()
Compacts multiple SSTable files into a single SSTable.
- std::string [search_sstables](#) (const std::string &key) const
Searches for a key in the SSTable files.
- void [flush](#) ()
Flushes the memtable to an SSTable file.
- void [compact](#) ()
Compacts multiple SSTable files into a single SSTable.
- std::string [search_sstables](#) (const std::string &key) const
Searches SSTable files for a given key.

Private Attributes

- std::unordered_map< std::string, [Entry](#) > [store](#)
In-memory memtable storing key-value entries.
- size_t [capacity](#)
Maximum entries allowed in the memtable.
- size_t [size](#)
Current number of entries in the memtable.
- std::vector< std::string > [sstables](#)
List of SSTable file names used for persistence.
- [LRUCache](#)< std::string, std::string > [cache](#)
LRU cache instance used for fast GET operations.

3.1.1 Detailed Description

[BlinkDB](#) is a lightweight database with an in-memory memtable and SSTable persistence.

[BlinkDB](#) is a lightweight key-value store that uses an in-memory memtable with SSTable persistence.

It supports basic operations (set, get, delete) and uses an LRU cache to speed up read operations.

It supports basic operations like SET, GET, and DEL and utilizes an LRU cache to accelerate GET operations.

3.1.2 Constructor & Destructor Documentation

3.1.2.1 [BlinkDB\(\)](#) [1/2]

```
BlinkDB::BlinkDB (
    size_t capacity = 64*1024*1024,
    size_t cache_capacity = 16*1024*1024 ) [inline], [explicit]
```

Constructs a [BlinkDB](#) instance.

Initializes the memtable with a given capacity and sets up the LRU cache with its own capacity.

Parameters

<i>capacity</i>	Maximum entries for the memtable (default: 64*1024*1024).
<i>cache_capacity</i>	Capacity for the LRU cache (default: 16*1024*1024).

3.1.2.2 [BlinkDB\(\)](#) [2/2]

```
BlinkDB::BlinkDB (
    size_t capacity = 64*1024*1024,
    size_t cache_capacity = 16*1024*1024 ) [inline], [explicit]
```

Constructs a [BlinkDB](#) instance.

Initializes the in-memory memtable with a given capacity and sets up the LRU cache with its capacity.

Parameters

<i>capacity</i>	Maximum number of entries in the memtable (default: 64*1024*1024).
<i>cache_capacity</i>	Capacity for the LRU cache (default: 16*1024*1024).

3.1.3 Member Function Documentation

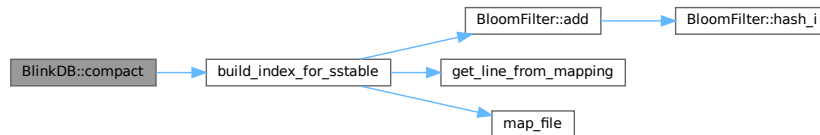
3.1.3.1 [compact\(\)](#) [1/2]

```
void BlinkDB::compact ( ) [private]
```

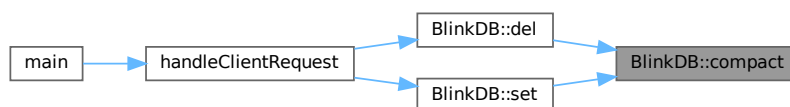
Compacts multiple SSTable files into a single SSTable.

This operation merges multiple SSTables, removes tombstones, and rebuilds indexes.

If the number of SSTable files exceeds a threshold (10), all files are merged. The merge process combines records, overwrites older entries with newer ones, removes tombstones, and rebuilds the in-memory indices, Bloom filters, and memory maps. Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.2 compact() [2/2]

```
void BlinkDB::compact ( ) [private]
```

Compacts multiple SSTable files into a single SSTable.

This process merges records from various SSTables, removing obsolete entries and tombstones.

3.1.3.3 del() [1/2]

```
bool BlinkDB::del (
    const std::string & key )
```

Deletes a key from the database.

Deletes a key from [BlinkDB](#).

Marks the key as deleted in the memtable and removes it from the cache.

Parameters

<i>key</i>	The key to delete.
------------	--------------------

Returns

true if the key was found and deleted; false otherwise.

The key is removed from the cache and marked as deleted in the in-memory store. If the key exists in any SSTable, it is also marked as a tombstone.

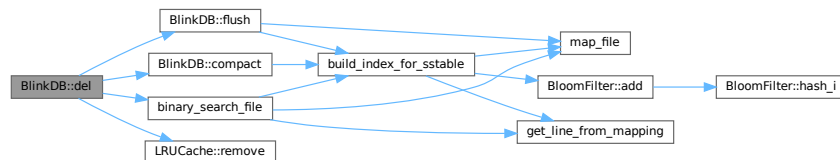
Parameters

<i>key</i>	The key to delete.
------------	--------------------

Returns

true if the key was found and deleted, false otherwise.

Here is the call graph for this function:



Here is the caller graph for this function:

**3.1.3.4 del() [2/2]**

```
bool BlinkDB::del (
    const std::string & key )
```

Deletes a key from the database.

The key is removed from the cache and marked as deleted in the memtable.

Parameters

<i>key</i>	The key to delete.
------------	--------------------

Returns

true if the key was found and marked as deleted; false otherwise.

3.1.3.5 flush() [1/2]

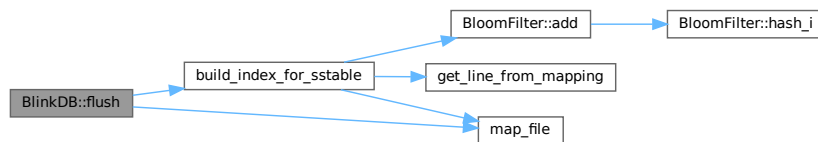
```
void BlinkDB::flush ( ) [private]
```

Flushes the memtable to an SSTable file.

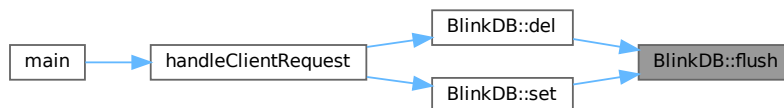
Flushes the in-memory store to a new SSTable file.

This operation writes all entries in the memtable to a disk file.

The in-memory store is written to disk, and a new SSTable is created. The in-memory index, Bloom filter, and memory map are built for the new file. Here is the call graph for this function:



Here is the caller graph for this function:

**3.1.3.6 flush() [2/2]**

```
void BlinkDB::flush ( ) [private]
```

Flushes the memtable to an SSTable file.

This operation writes all entries from the memtable to disk.

3.1.3.7 get() [1/2]

```
std::string BlinkDB::get (
    const std::string & key )
```

Retrieves the value associated with a given key.

Retrieves the value associated with a key in [BlinkDB](#).

First checks the LRU cache and then the memtable and SSTables.

Parameters

<i>key</i>	The key to retrieve.
------------	----------------------

Returns

The value associated with the key, or "NULL" if the key is not found.

The method first checks the LRU cache, then the in-memory store, and finally searches the SSTable files using an optimized lookup.

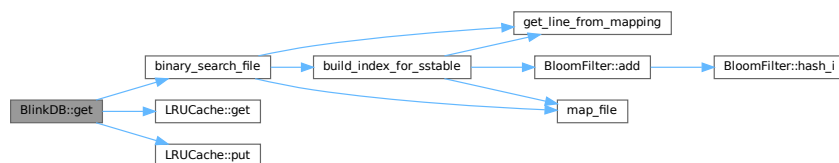
Parameters

<i>key</i>	The key to retrieve.
------------	----------------------

Returns

The associated value, or "NULL" if the key is not found or marked as deleted.

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.8 get() [2/2]

```
std::string BlinkDB::get (
    const std::string & key )
```

Retrieves the value associated with a key.

The function first checks the LRU cache, then the memtable, and finally the SSTables.

Parameters

<i>key</i>	The key to retrieve.
------------	----------------------

Returns

The associated value, or "NULL" if the key is not found.

3.1.3.9 search_sstables() [1/2]

```
std::string BlinkDB::search_sstables (
    const std::string & key ) const [private]
```

Searches for a key in the SSTable files.

Searches SSTable files for a given key.

Searches from the newest to the oldest SSTable file using an optimized lookup.

Parameters

<i>key</i>	The key to search for.
------------	------------------------

Returns

The value associated with the key if found; otherwise "NULL".

The method searches from the newest to the oldest SSTable using the optimized lookup.

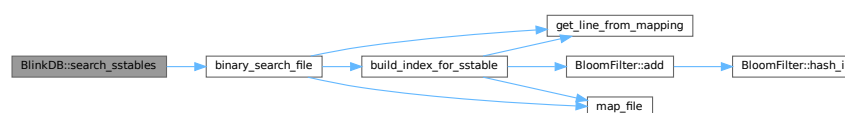
Parameters

<i>key</i>	The key to search for.
------------	------------------------

Returns

The associated value, or "NULL" if not found.

Here is the call graph for this function:



3.1.3.10 search_sstables() [2/2]

```
std::string BlinkDB::search_sstables (
    const std::string & key ) const [private]
```

Searches SSTable files for a given key.

The search is performed from the newest to the oldest SSTable.

Parameters

<i>key</i>	The key to search for.
------------	------------------------

Returns

The associated value if found; otherwise "NULL".

3.1.3.11 set() [1/2]

```
void BlinkDB::set (
    const std::string & key,
    const std::string & value )
```

Sets a key-value pair in the database.

Sets a key-value pair in [BlinkDB](#).

If the key already exists, its value is updated.

Parameters

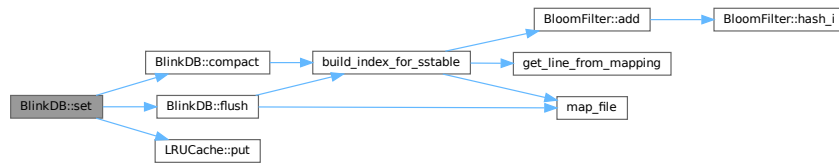
<i>key</i>	The key to set.
<i>value</i>	The value to associate with the key.

If the key exists, its value is updated and marked as not deleted. If adding a new key and capacity is reached, the in-memory store is flushed and compacted.

Parameters

<i>key</i>	The key to set.
<i>value</i>	The value to associate with the key.

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.12 set() [2/2]

```
void BlinkDB::set (
    const std::string & key,
    const std::string & value )
```

Sets a key-value pair in the database.

If the key exists, its value is updated. If the memtable is full, it is flushed to disk.

Parameters

<i>key</i>	The key to set.
<i>value</i>	The value to associate with the key.

3.1.4 Member Data Documentation

3.1.4.1 cache

```
LRUCache< std::string, std::string > BlinkDB::cache [private]
```

LRU cache instance used for fast GET operations.

LRU cache instance to optimize GET operations.

Caches key-value pairs to improve performance.

3.1.4.2 capacity

```
size_t BlinkDB::capacity [private]
```

Maximum entries allowed in the memtable.

Maximum number of entries allowed in the memtable.

3.1.4.3 size

```
size_t BlinkDB::size [private]
```

Current number of entries in the memtable.

3.1.4.4 sstables

```
std::vector< std::string > BlinkDB::sstables [private]
```

List of SSTable file names used for persistence.

List of SSTable file names for persistence.

3.1.4.5 store

```
std::unordered_map< std::string, Entry > BlinkDB::store [private]
```

In-memory memtable storing key-value entries.

In-memory memtable for key-value entries.

The documentation for this class was generated from the following files:

- [blinkdb.h](#)
- [server.cpp](#)
- [blinkdb.cpp](#)

3.2 BloomFilter Class Reference

A simple Bloom filter implementation.

Public Member Functions

- [BloomFilter](#) (size_t m, size_t k)
Constructs a [BloomFilter](#).
- void [add](#) (const std::string &s)
Adds an element to the Bloom filter.
- bool [contains](#) (const std::string &s) const
Checks if an element might be in the Bloom filter.

Private Member Functions

- `size_t hash_i` (`const std::string &s, size_t i`) `const`
Generates a hash value for a given string and hash function index.

Private Attributes

- `size_t m_bits`
The number of bits in the Bloom filter.
- `size_t k_hashes`
The number of hash functions.
- `std::vector< bool > bits`
The bit vector representing the filter.

3.2.1 Detailed Description

A simple Bloom filter implementation.

This class implements a basic Bloom filter using a vector of booleans.

3.2.2 Constructor & Destructor Documentation

3.2.2.1 BloomFilter()

```
BloomFilter::BloomFilter (
    size_t m,
    size_t k ) [inline]
```

Constructs a [BloomFilter](#).

Parameters

<i>m</i>	The number of bits in the filter.
<i>k</i>	The number of hash functions.

3.2.3 Member Function Documentation

3.2.3.1 add()

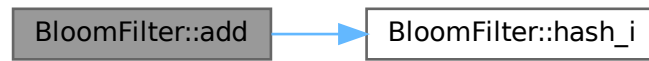
```
void BloomFilter::add (
    const std::string & s ) [inline]
```

Adds an element to the Bloom filter.

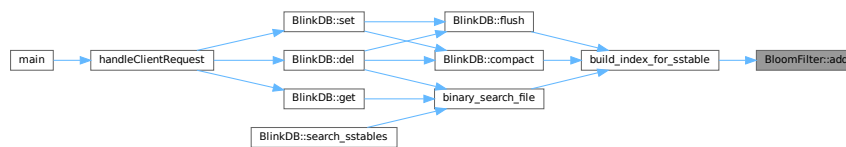
Parameters

<i>s</i>	The element (as a string) to add.
----------	-----------------------------------

Here is the call graph for this function:



Here is the caller graph for this function:



3.2.3.2 contains()

```
bool BloomFilter::contains (
    const std::string & s ) const [inline]
```

Checks if an element might be in the Bloom filter.

Parameters

<code>s</code>	The element (as a string) to check.
----------------	-------------------------------------

Returns

true if the element might be in the filter, false if it definitely is not.

Here is the call graph for this function:



3.2.3.3 hash_i()

```
size_t BloomFilter::hash_i (
    const std::string & s,
    size_t i ) const [inline], [private]
```

Generates a hash value for a given string and hash function index.

This combines the standard `std::hash` with a seed value.

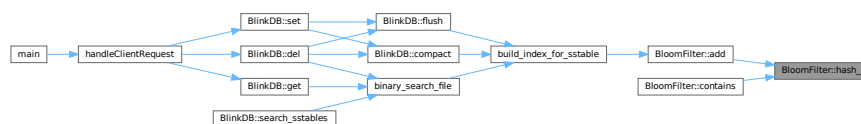
Parameters

<i>s</i>	The input string.
<i>i</i>	The index of the hash function.

Returns

The generated hash value.

Here is the caller graph for this function:



3.2.4 Member Data Documentation

3.2.4.1 bits

```
std::vector<bool> BloomFilter::bits [private]
```

The bit vector representing the filter.

3.2.4.2 k_hashes

```
size_t BloomFilter::k_hashes [private]
```

The number of hash functions.

3.2.4.3 m_bits

```
size_t BloomFilter::m_bits [private]
```

The number of bits in the Bloom filter.

The documentation for this class was generated from the following file:

- [blinkdb.cpp](#)

3.3 BlinkDB::Entry Struct Reference

Structure to store a value and its deletion state.

Public Attributes

- `std::string value`
The stored value.
- `bool deleted = false`
Flag indicating if the entry has been marked as deleted.

3.3.1 Detailed Description

Structure to store a value and its deletion state.

Represents an entry in the in-memory memtable.

3.3.2 Member Data Documentation

3.3.2.1 deleted

```
bool BlinkDB::Entry::deleted = false
```

Flag indicating if the entry has been marked as deleted.

Flag indicating if the entry is marked as deleted.

3.3.2.2 value

```
std::string BlinkDB::Entry::value
```

The stored value.

The documentation for this struct was generated from the following files:

- [blinkdb.h](#)
- [server.cpp](#)

3.4 LRUCache< K, V > Class Template Reference

A templated Least Recently Used (LRU) Cache.

```
#include <lru_cache.h>
```

Public Member Functions

- [LRUCache](#) (size_t [capacity](#))
Constructs an [LRUCache](#) with a specified capacity.
- bool [get](#) (const K &key, V &value)
Retrieves a value by key from the cache.
- void [put](#) (const K &key, const V &value)
Inserts or updates a key-value pair in the cache.
- void [remove](#) (const K &key)
Removes a key from the cache.

Private Attributes

- size_t [capacity](#)
Maximum capacity of the cache.
- std::list< std::pair< K, V > > [cacheList](#)
List to maintain order of usage.
- std::unordered_map< K, typename std::list< std::pair< K, V > >::iterator > [cacheMap](#)
Map to store keys and corresponding list iterators.

3.4.1 Detailed Description

```
template<typename K, typename V>
class LRUCache< K, V >
```

A templated Least Recently Used (LRU) Cache.

This class implements an LRU cache using a combination of an unordered_map and a doubly-linked list. The cache evicts the least recently used item when its capacity is reached.

Template Parameters

<i>K</i>	Type for the keys.
<i>V</i>	Type for the values.

3.4.2 Constructor & Destructor Documentation

3.4.2.1 LRUCache()

```
template<typename K , typename V >
LRUCache< K, V >::LRUCache (
    size_t capacity ) [inline], [explicit]
```

Constructs an [LRUCache](#) with a specified capacity.

Parameters

<i>capacity</i>	The maximum number of elements the cache can hold.
-----------------	--

3.4.3 Member Function Documentation

3.4.3.1 get()

```
template<typename K , typename V >
bool LRUCache< K, V >::get (
    const K & key,
    V & value ) [inline]
```

Retrieves a value by key from the cache.

If the key is found, the corresponding item is moved to the front (most recently used) and the value is returned via the output parameter.

Parameters

<i>key</i>	The key to retrieve.
<i>value</i>	Output parameter that will contain the retrieved value.

Returns

true if the key was found, false otherwise.

Here is the caller graph for this function:



3.4.3.2 put()

```
template<typename K , typename V >
void LRUCache< K, V >::put (
    const K & key,
    const V & value ) [inline]
```

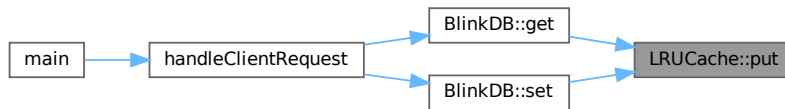
Inserts or updates a key-value pair in the cache.

If the key exists, its value is updated and the item is moved to the front. If the key does not exist and the cache is full, the least recently used item is removed before the new key-value pair is inserted.

Parameters

<i>key</i>	The key to insert or update.
<i>value</i>	The value to associate with the key.

Here is the caller graph for this function:



3.4.3.3 remove()

```

template<typename K , typename V >
void LRUCache< K, V >::remove (
    const K & key ) [inline]
  
```

Removes a key from the cache.

If the key exists in the cache, it is removed.

Parameters

<i>key</i>	The key to remove.
------------	--------------------

Here is the caller graph for this function:



3.4.4 Member Data Documentation

3.4.4.1 cacheList

```

template<typename K , typename V >
std::list<std::pair<K, V> > LRUCache< K, V >::cacheList [private]
  
```

List to maintain order of usage.

3.4.4.2 cacheMap

```

template<typename K , typename V >
std::unordered_map<K, typename std::list<std::pair<K, V> >::iterator> LRUCache< K, V >::cacheMap [private]
  
```

Map to store keys and corresponding list iterators.

3.4.4.3 capacity

```
template<typename K , typename V >
size_t LRUCache< K, V >::capacity [private]
```

Maximum capacity of the cache.

The documentation for this class was generated from the following file:

- [lru_cache.h](#)

3.5 RESPParser Class Reference

A simple RESP (Redis Serialization Protocol) parser and serializer.

Static Public Member Functions

- static std::vector< std::string > [parse](#) (const std::string &input)
Parses a RESP-formatted input string into tokens.
- static std::string [serialize](#) (const std::string &data)
Serializes data into RESP format.
- static std::string [ok](#) ()
Generates a RESP "OK" message.
- static std::string [error](#) (const std::string &msg)
Generates a RESP error message.

3.5.1 Detailed Description

A simple RESP (Redis Serialization Protocol) parser and serializer.

This class provides static methods to parse RESP input and serialize responses.

3.5.2 Member Function Documentation

3.5.2.1 error()

```
static std::string RESPParser::error (
    const std::string & msg ) [inline], [static]
```

Generates a RESP error message.

Parameters

<i>msg</i>	The error message.
------------	--------------------

Returns

A RESP-formatted error message.

Here is the caller graph for this function:

**3.5.2.2 ok()**

```
static std::string RESPParser::ok ( ) [inline], [static]
```

Generates a RESP "OK" message.

Returns

A RESP-formatted "OK" message.

Here is the caller graph for this function:

**3.5.2.3 parse()**

```
static std::vector< std::string > RESPParser::parse (
    const std::string & input ) [inline], [static]
```

Parses a RESP-formatted input string into tokens.

The function expects the input to be in RESP format and extracts tokens accordingly.

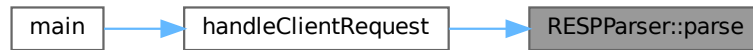
Parameters

<i>input</i>	The RESP-formatted string.
--------------	----------------------------

Returns

A vector of tokens extracted from the input.

Here is the caller graph for this function:

**3.5.2.4 serialize()**

```
static std::string RESPParser::serialize (  
    const std::string & data ) [inline], [static]
```

Serializes data into RESP format.

For "NULL" data, a special RESP representation is returned.

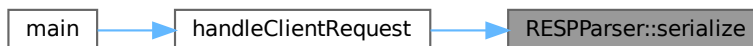
Parameters

<i>data</i>	The data to serialize.
-------------	------------------------

Returns

A RESP-formatted string representing the data.

Here is the caller graph for this function:



The documentation for this class was generated from the following file:

- [server.cpp](#)

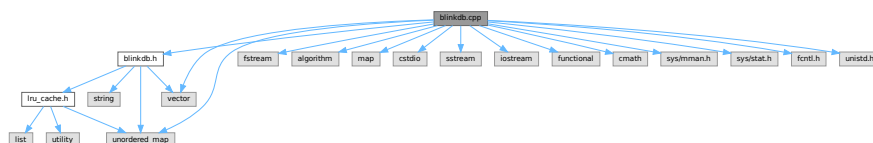
Chapter 4

File Documentation

4.1 blinkdb.cpp File Reference

```
#include "blinkdb.h"
#include <fstream>
#include <algorithm>
#include <vector>
#include <map>
#include <cstdio>
#include <sstream>
#include <iostream>
#include <unordered_map>
#include <functional>
#include <cmath>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
```

Include dependency graph for blinkdb.cpp:



Classes

- class [BloomFilter](#)
A simple Bloom filter implementation.

Functions

- static const char * [map_file](#) (const std::string &filename, size_t &filesize)
Memory-maps a file and returns a pointer to its contents along with the file size.
- static std::string [get_line_from_mapping](#) (const char *data, size_t filesize, size_t offset)
Retrieves a full line from a memory-mapped file given an offset.
- static std::vector< std::pair< std::string, std::streampos > > [build_index_for_sstable](#) (const std::string &filename)
Builds an in-memory index and Bloom filter for an SSTable file.
- static std::string [binary_search_file](#) (const std::string &filename, const std::string &key)
Performs a binary search on the in-memory index for a given SSTable file.

Variables

- static std::unordered_map< std::string, std::vector< std::pair< std::string, std::streampos > > > [sstable_indices](#)
- static std::unordered_map< std::string, [BloomFilter](#) > [sstable_filters](#)
- static std::unordered_map< std::string, std::pair< const char *, size_t > > [sstable_mmaps](#)

4.1.1 Function Documentation

4.1.1.1 [binary_search_file\(\)](#)

```
static std::string binary_search_file (
    const std::string & filename,
    const std::string & key ) [static]
```

Performs a binary search on the in-memory index for a given SSTable file.

First checks the Bloom filter to quickly rule out missing keys, then performs a binary search on the index and retrieves the record from the memory-mapped file.

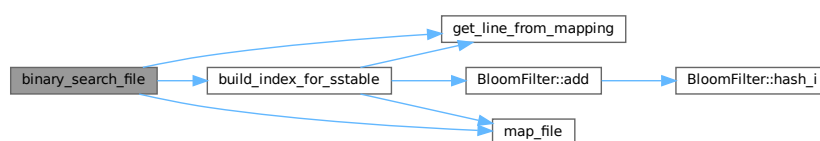
Parameters

<i>filename</i>	The SSTable file name.
<i>key</i>	The key to search for.

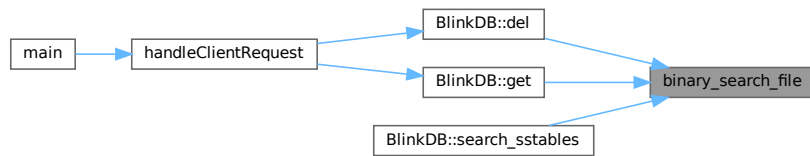
Returns

The value corresponding to the key, or "NULL" if not found or if it's a tombstone.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.1.2 build_index_for_sstable()

```
static std::vector< std::pair< std::string, std::streampos > > build_index_for_sstable (
    const std::string & filename ) [static]
```

Builds an in-memory index and Bloom filter for an SSTable file.

It reads each line (record) from the file, extracts the key and file offset, and then builds both the sorted index and the Bloom filter.

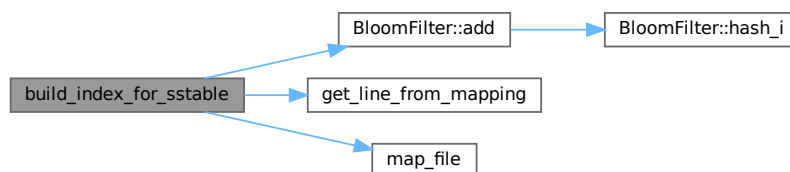
Parameters

<i>filename</i>	The SSTable file name.
-----------------	------------------------

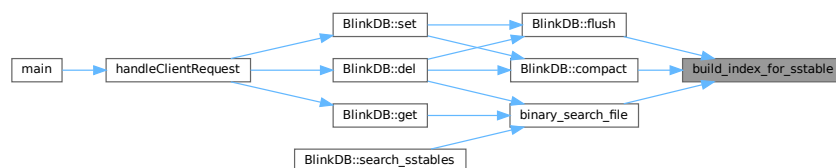
Returns

A sorted vector of (key, file offset) pairs.

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.1.3 get_line_from_mapping()

```
static std::string get_line_from_mapping (
    const char * data,
    size_t filesize,
    size_t offset ) [static]
```

Retrieves a full line from a memory-mapped file given an offset.

The returned string does not include the newline character.

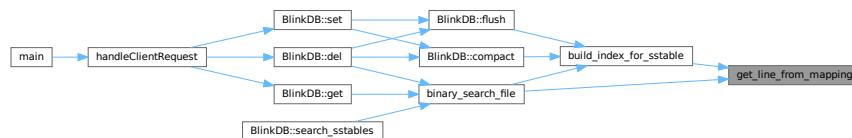
Parameters

<i>data</i>	Pointer to the memory-mapped file.
<i>filesize</i>	The size of the file.
<i>offset</i>	The offset from where to start reading.

Returns

The line as a string.

Here is the caller graph for this function:



4.1.1.4 map_file()

```
static const char * map_file (
    const std::string & filename,
    size_t & filesize ) [static]
```

Memory-maps a file and returns a pointer to its contents along with the file size.

If the file is already memory-mapped, it returns the existing mapping.

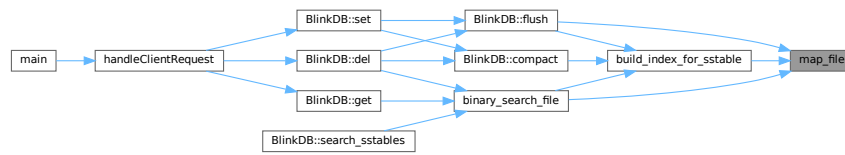
Parameters

<i>filename</i>	The name of the file to map.
<i>filesize</i>	Output parameter that will contain the size of the file.

Returns

Pointer to the mapped file data, or nullptr on failure.

Here is the caller graph for this function:



4.1.2 Variable Documentation

4.1.2.1 sstable_filters

```
std::unordered_map<std::string, BloomFilter> sstable_filters [static]
```

4.1.2.2 sstable_indices

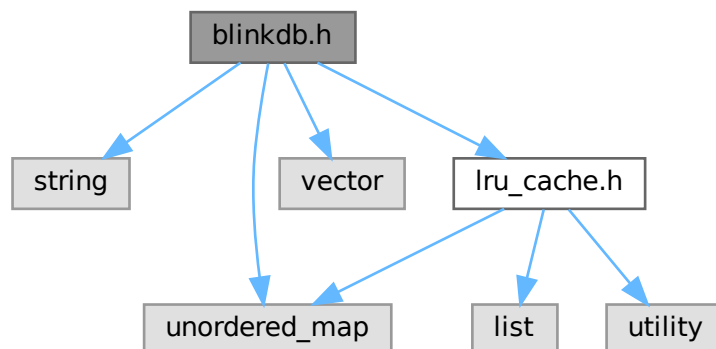
```
std::unordered_map<std::string, std::vector<std::pair<std::string, std::streampos> > > > sstable←
_indices [static]
```

4.1.2.3 sstable_mmmaps

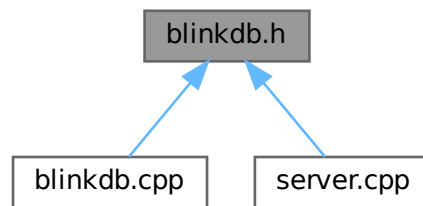
```
std::unordered_map<std::string, std::pair<const char*, size_t> > sstable_mmmaps [static]
```

4.2 blinkdb.h File Reference

```
#include <string>
#include <unordered_map>
#include <vector>
#include "lru_cache.h"
Include dependency graph for blinkdb.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [BlinkDB](#)
BlinkDB is a lightweight database with an in-memory memtable and SSTable persistence.
- struct [BlinkDB::Entry](#)
Structure to store a value and its deletion state.

4.3 blinkdb.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include <string>
00003 #include <unordered_map>
00004 #include <vector>
00005 #include "lru_cache.h" // include the LRU cache header
00006
00012 class BlinkDB
00013 {
00014 private:
00018     struct Entry
00019     {
00020         std::string value;
00021         bool deleted = false;
00022     };
00023
00024     std::unordered_map<std::string, Entry> store;
00025     size_t capacity;
00026     size_t size;
00027     std::vector<std::string> sstables;
00028
00034     void flush();
00035
00041     void compact();
00042
00051     std::string search_sstables(const std::string &key) const;
00052
00058     LRUCache<std::string, std::string> cache;
00059
00060 public:
00069     explicit BlinkDB(size_t capacity = 64*1024*1024, size_t cache_capacity = 16*1024*1024)
00070         : capacity(capacity), size(0), cache(cache_capacity) {}
00071
00080     void set(const std::string &key, const std::string &value);
00081
00090     std::string get(const std::string &key);
00091
00100     bool del(const std::string &key);
00101 };
  
```

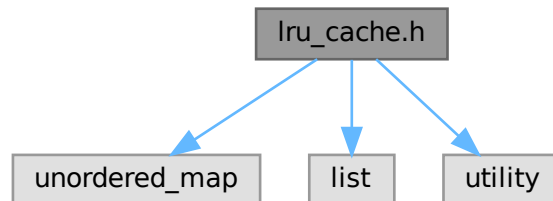
4.4 lru_cache.h File Reference

```
#include <unordered_map>
```

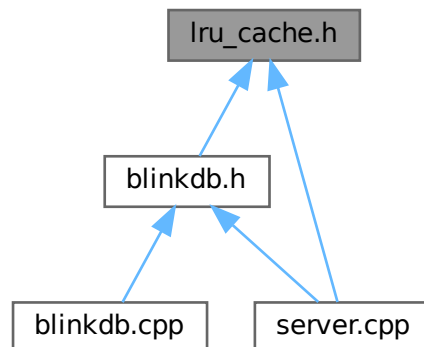
```
#include <list>
```

```
#include <utility>
```

Include dependency graph for lru_cache.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [LRUCache< K, V >](#)

A templated Least Recently Used (LRU) Cache.

4.5 lru_cache.h

[Go to the documentation of this file.](#)

```
00001 #ifndef LRUCACHE_H
00002 #define LRUCACHE_H
```

```

00003
00004 #include <unordered_map>
00005 #include <list>
00006 #include <utility>
00007
00018 template<typename K, typename V>
00019 class LRUCache {
00020 public:
00026     explicit LRUCache(size_t capacity) : capacity(capacity) {}
00027
00038     bool get(const K &key, V &value) {
00039         auto it = cacheMap.find(key);
00040         if (it == cacheMap.end())
00041             return false;
00042         // Move accessed item to the front (most recently used)
00043         cacheList.splice(cacheList.begin(), cacheList, it->second);
00044         value = it->second->second;
00045         return true;
00046     }
00047
00058     void put(const K &key, const V &value) {
00059         auto it = cacheMap.find(key);
00060         if (it != cacheMap.end()) {
00061             // Update value and move to front
00062             it->second->second = value;
00063             cacheList.splice(cacheList.begin(), cacheList, it->second);
00064             return;
00065         }
00066         // Check capacity, remove least recently used if needed
00067         if (cacheList.size() >= capacity) {
00068             auto last = cacheList.back();
00069             cacheMap.erase(last.first);
00070             cacheList.pop_back();
00071         }
00072         cacheList.emplace_front(key, value);
00073         cacheMap[key] = cacheList.begin();
00074     }
00075
00083     void remove(const K &key) {
00084         auto it = cacheMap.find(key);
00085         if (it != cacheMap.end()) {
00086             cacheList.erase(it->second);
00087             cacheMap.erase(it);
00088         }
00089     }
00090
00091 private:
00092     size_t capacity;
00093     std::list<std::pair<K, V> > cacheList;
00094     std::unordered_map<K, typename std::list<std::pair<K, V>>::iterator> cacheMap;
00095 };
00096
00097 #endif // LRUCACHE_H

```

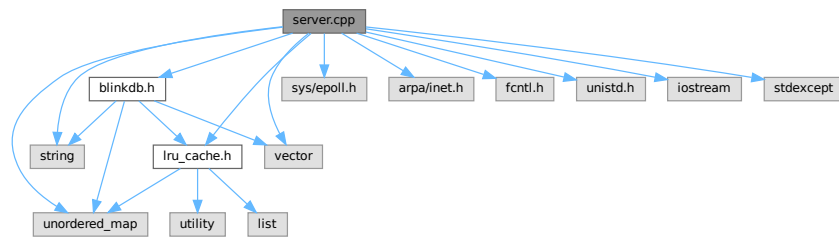
4.6 server.cpp File Reference

```

#include <string>
#include <unordered_map>
#include <vector>
#include "lru_cache.h"
#include "blinkdb.h"
#include <sys/epoll.h>
#include <arpa/inet.h>
#include <fcntl.h>
#include <unistd.h>
#include <iostream>
#include <stdexcept>

```

Include dependency graph for server.cpp:



Classes

- class [BlinkDB](#)
BlinkDB is a lightweight database with an in-memory memtable and SSTable persistence.
- struct [BlinkDB::Entry](#)
Structure to store a value and its deletion state.
- class [RESPParser](#)
A simple RESP (REdis Serialization Protocol) parser and serializer.

Macros

- #define [MAX_EVENTS](#) 10
Maximum number of events to process per epoll_wait call.
- #define [BUFFER_SIZE](#) 4096
Buffer size for reading client data.
- #define [PORT](#) 9001
Port number for the server to listen on.

Functions

- void [handleClientRequest](#) (int clientFd, [BlinkDB](#) &db)
Handles an individual client request.
- int [main](#) ()
Entry point for the [BlinkDB](#) server application.

4.6.1 Macro Definition Documentation

4.6.1.1 BUFFER_SIZE

```
#define BUFFER_SIZE 4096
```

Buffer size for reading client data.

4.6.1.2 MAX_EVENTS

```
#define MAX_EVENTS 10
```

Maximum number of events to process per `epoll_wait` call.

4.6.1.3 PORT

```
#define PORT 9001
```

Port number for the server to listen on.

4.6.2 Function Documentation

4.6.2.1 handleClientRequest()

```
void handleClientRequest (
    int clientFd,
    BlinkDB & db )
```

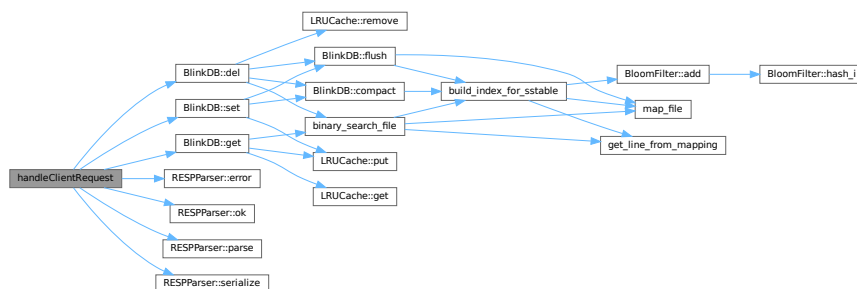
Handles an individual client request.

Reads the incoming data, parses the RESP command, executes it against [BlinkDB](#), and sends back the appropriate response.

Parameters

<i>clientFd</i>	The file descriptor for the client connection.
<i>db</i>	A reference to the BlinkDB instance.

Here is the call graph for this function:



Here is the caller graph for this function:



4.6.2.2 main()

```
int main ( )
```

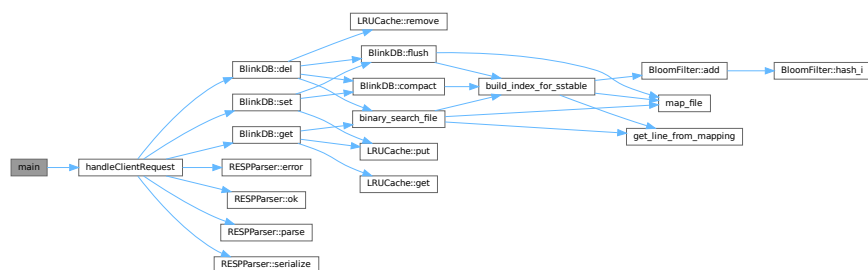
Entry point for the [BlinkDB](#) server application.

Sets up a TCP server, initializes epoll for I/O multiplexing, and handles client connections.

Returns

int Exit status.

Here is the call graph for this function:



Index

- add
 - BloomFilter, [16](#)
- binary_search_file
 - blinkdb.cpp, [28](#)
- bits
 - BloomFilter, [18](#)
- BlinkDB, [5](#)
 - BlinkDB, [7](#)
 - cache, [14](#)
 - capacity, [14](#)
 - compact, [7](#), [8](#)
 - del, [8](#), [9](#)
 - flush, [10](#)
 - get, [10](#), [11](#)
 - search_sstables, [12](#)
 - set, [13](#), [14](#)
 - size, [15](#)
 - sstables, [15](#)
 - store, [15](#)
- blinkdb.cpp, [27](#)
 - binary_search_file, [28](#)
 - build_index_for_sstable, [29](#)
 - get_line_from_mapping, [30](#)
 - map_file, [30](#)
 - sstable_filters, [31](#)
 - sstable_indices, [31](#)
 - sstable_mmaps, [31](#)
- blinkdb.h, [31](#)
- BlinkDB::Entry, [19](#)
 - deleted, [19](#)
 - value, [19](#)
- BloomFilter, [15](#)
 - add, [16](#)
 - bits, [18](#)
 - BloomFilter, [16](#)
 - contains, [17](#)
 - hash_i, [17](#)
 - k_hashes, [18](#)
 - m_bits, [18](#)
- BUFFER_SIZE
 - server.cpp, [35](#)
- build_index_for_sstable
 - blinkdb.cpp, [29](#)
- cache
 - BlinkDB, [14](#)
- cacheList
 - LRUCache< K, V >, [22](#)
- cacheMap
 - LRUCache< K, V >, [22](#)
- capacity
 - BlinkDB, [14](#)
 - LRUCache< K, V >, [22](#)
- compact
 - BlinkDB, [7](#), [8](#)
- contains
 - BloomFilter, [17](#)
- del
 - BlinkDB, [8](#), [9](#)
- deleted
 - BlinkDB::Entry, [19](#)
- error
 - RESPParser, [23](#)
- flush
 - BlinkDB, [10](#)
- get
 - BlinkDB, [10](#), [11](#)
 - LRUCache< K, V >, [21](#)
- get_line_from_mapping
 - blinkdb.cpp, [30](#)
- handleClientRequest
 - server.cpp, [36](#)
- hash_i
 - BloomFilter, [17](#)
- k_hashes
 - BloomFilter, [18](#)
- lru_cache.h, [33](#)
- LRUCache
 - LRUCache< K, V >, [20](#)
- LRUCache< K, V >, [19](#)
 - cacheList, [22](#)
 - cacheMap, [22](#)
 - capacity, [22](#)
 - get, [21](#)
 - LRUCache, [20](#)
 - put, [21](#)
 - remove, [22](#)
- m_bits
 - BloomFilter, [18](#)
- main
 - server.cpp, [37](#)
- map_file

- blinkdb.cpp, [30](#)
- MAX_EVENTS
 - server.cpp, [35](#)
- ok
 - RESPParser, [24](#)
- parse
 - RESPParser, [24](#)
- PORT
 - server.cpp, [36](#)
- put
 - LRUCache< K, V >, [21](#)
- remove
 - LRUCache< K, V >, [22](#)
- RESPParser, [23](#)
 - error, [23](#)
 - ok, [24](#)
 - parse, [24](#)
 - serialize, [25](#)
- search_sstables
 - BlinkDB, [12](#)
- serialize
 - RESPParser, [25](#)
- server.cpp, [34](#)
 - BUFFER_SIZE, [35](#)
 - handleClientRequest, [36](#)
 - main, [37](#)
 - MAX_EVENTS, [35](#)
 - PORT, [36](#)
- set
 - BlinkDB, [13](#), [14](#)
- size
 - BlinkDB, [15](#)
- sstable_filters
 - blinkdb.cpp, [31](#)
- sstable_indices
 - blinkdb.cpp, [31](#)
- sstable_mmaps
 - blinkdb.cpp, [31](#)
- sstables
 - BlinkDB, [15](#)
- store
 - BlinkDB, [15](#)
- value
 - BlinkDB::Entry, [19](#)